

Глава 2. SPL, GPIO и немного хардвара

Ежелев Г. И.

18 июля 2026 г.

В предыдущей главе мы пользовались библиотекой CMSIS - Cortex Microcontroller Software Interface Standard.

Подобные библиотеки есть для всех МК, в том числе и ардуино. Везде один и тот же принцип - настройка всего через регистры. Более того, любая программа для любого ПК в итоге сводиться к схожему набору действий. Однако для STM32 есть еще 2 библиотеки для работы с периферией: SPL - Standard Peripheral Library и HAL - Hardware Abstraction Layer. HAL самая высокоуровневая, SPL чуть ближе к CMSIS. SPL и HAL под капотом используют CMSIS.

SPL по факту сопоставляет каждому значению каждого параметра свой набор битов в регистре, причем SPL будет подстраиваться под выбранный МК (у разных МК могут по разному работать регистры с одинаковым названием и назначением), что ускорит переход с одной STM на другую.

Устройство SPL

Для каждого устройства внутри STM32 в библиотеке SPL есть своя **структура (structure)**. Загуглите, что такое structure в C++.

Она содержит **значение каждого параметра** устройства (Весь набор значений одного параметра часто тоже является структурой).

Когда мы настраиваем какое-то устройство, сначала создаем экземпляр структуры, далее заполняем **поля структуры** и далее передаем заполненную структуру в **функцию инициализации структуры**.

Разберем на примере GPIO

STM32

```
typedef struct
```

```
{
    uint32_t GPIO_Pin;           /*!< Specifies the GPIO pins to be configured.
                                   This parameter can be any value of @ref GPIO_pins_define */

    GPIOMode_TypeDef GPIO_Mode;  /*!< Specifies the operating mode for the selected pins.
                                   This parameter can be a value of @ref GPIOMode_TypeDef */

    GPIOSpeed_TypeDef GPIO_Speed; /*!< Specifies the speed for the selected pins.
                                   This parameter can be a value of @ref GPIOSpeed_TypeDef */

    GPIOType_TypeDef GPIO_OType; /*!< Specifies the operating output type for the selected pins.
                                   This parameter can be a value of @ref GPIOType_TypeDef */

    GPIOPuPd_TypeDef GPIO_PuPd;  /*!< Specifies the operating Pull-up/Pull down for the selected pins.
                                   This parameter can be a value of @ref GPIOPuPd_TypeDef */
}GPIO_InitTypeDef;
```

Эту структуру можно найти в файле stm32f4xx_gpio.h в разделе spl4.

Слева параметры, справа их описание. После слова @ref (reference) идет текст, который есть перед структурой, содержащей **все возможные значения этого параметра**, при поиске текста после @ref через **ctrl+f** в этом файле вы найдете все допустимые значения параметра. Просто исследуя этот файл можно полностью понять принцип работы любого устройства и его настроить.

ава 2.

SPL

ев Г. И.

Пример настройки GPIO

STM32

Глава 2.

SPL

Ежелев Г. И.

```
1  GPIO_InitTypeDef _initParams;  
2  
3  _initParams.GPIO_Pin = GPIO_Pin_1;  
4  
5  _initParams.GPIO_Mode = GPIO_Mode_OUT;  
6  _initParams.GPIO_Speed = GPIO_Speed_100MHz;  
7  _initParams.GPIO_OType = GPIO_OType_PP;  
8  _initParams.GPIO_PuPd = GPIO_PuPd_DOWN;  
9  
10 GPIO_Init(GPIOA, &_initParams);
```

Попробуйте самостоятельно изучить за что отвечает каждый параметр. Можно гуглить.

Отдельно отметим строчку 1 - создание структуры, строчку 10 - инициализация пина GPIO по заполненной структуре и строчку 7 выбор режима подтяжки.

Пин может быть подтянут к земле(вниз) или к питанию (вверх), за это отвечает параметр PuPd. В обоих случаях подтяжка происходит либо через резистор - PP, либо напрямую OD (Open Drain). Второй случай не безопасен - возможно короткое замыкание, такое подключение используется редко, если вам нужна просто подтяжка **всегда выбирайте PP.**

Read & Write

GPIO_SetBits(GPIOA, GPIO_Pin_1); или

GPIO_SetBits(GPIOA, _initParams.GPIO_Pin); -

выставление единицы

GPIO_ResetBits(GPIOA, GPIO_Pin_1); - выставление 0

GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_1) - чтение
пина PA1

GPIO_ReadInputData(GPIOA) - чтение всего порта GPIOA,
эквивалентно чтению всего регистра IDR

Остальные функции так же можно найти в файле
stm32f4xx_gpio.h раздел spl4.

Очевидно SPL удобнее, чем CMSIS, поэтому далее мы
будем пользоваться преимущественно SPL (за
исключением главы 3).

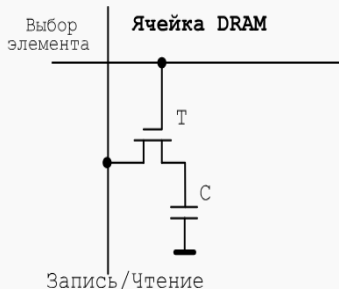
Память. DRAM

STM32

Глава 2.

SPL

Ежелев Г. И.



Ячейка **DRAM** - **Dynamic Random Access Memory** состоит из емкости и транзистора.

Конструкция довольно простая => память дешевая

Главный минус - постепенная **утечка заряда** с емкости.

Решает эту проблему **устройство регенерации памяти**, которое раз в $\approx 64\text{мс}$ читает значение с каждой ячейки и перезаписывает его, для обновления заряда. Так устроена **ОЗУ**.

Защелки. D-триггеры. Регистры

Перед тем как продолжить, попробуйте построить схему логического И, ИЛИ, НЕ

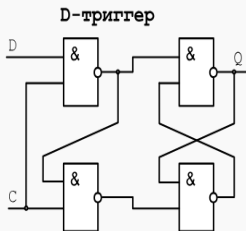
В самом ядре так же находятся свои маленькие ячейки памяти для хранения промежуточных результатов и управления устройствами - регистры. Они состоят из защелок (триггеров), обычно это D-триггеры.

D-триггер имеет 2 входа - Data и Clock. Data выставляет значение, Clock определяет, когда значение с входа D будет записано в триггер. Выход отвечает за хранимое значение.

Регистр - объединение D-триггеров с общим выводом Clock. Это самый быстрый (по скорости доступа) тип памяти в любой ЭВМ.

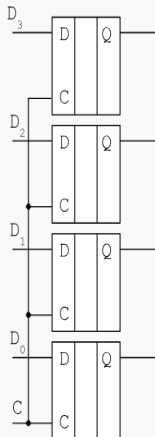


- Элементы хранения (триггеры, регистры)

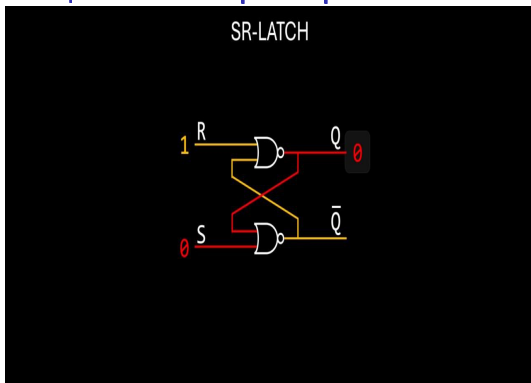


15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

0 1 1 0 0 0 1 0 0 0 0 0 1 0 1 1

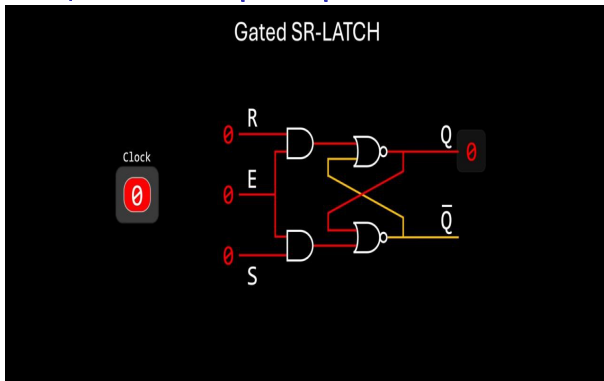


Защелки. SR-триггеры.



Такая защелка называется **SR-триггер** - **Set-Reset триггер**. То есть отдельный канал для записи единицы и для записи 0. попробуйте самостоятельно проследить, как происходит переключение сигналов. Что будет, если подать на обе шины 0? 1?

Защелки. SR-триггеры.



Дополним его сигналом **Enable** и подключим его к тактовому генератору (Clock). Таким образом мы исключили доступ к триггеру **вне периода импульса тактового генератора**. Обозначим полученную схему отдельным элементом.

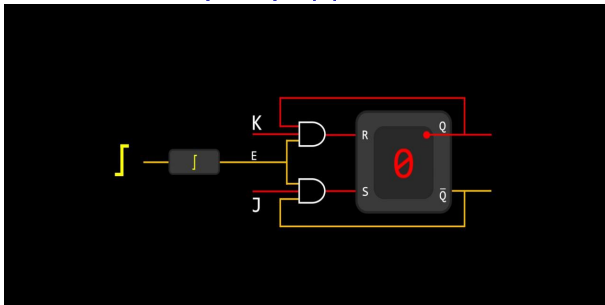
Защелки. Flip-flop. Делитель частоты

STM32

Глава 2.

SPL

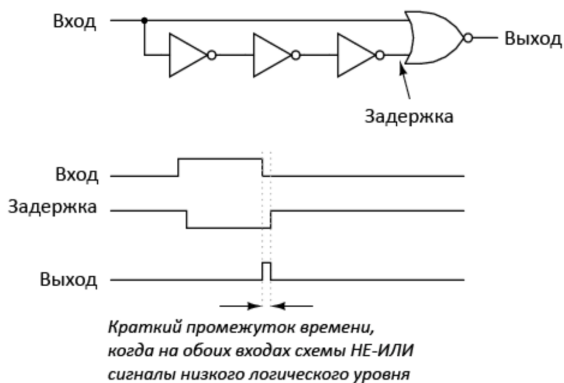
Ежелев Г. И.



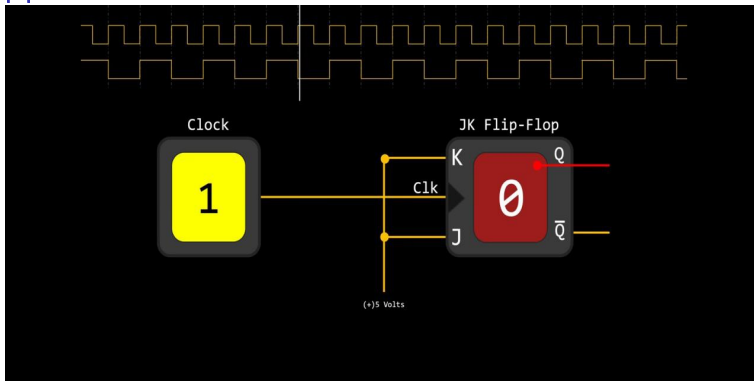
Подключим через **логическое И** соответствующие выходы триггера к входам. Enable подключим к тактовому генератору через **детектор переднего фронта**. Обозначим полученную схему - **Flip-Flop** - как один элемент. Входы обозначим буквами J и K. Отметим, что теперь мы можем **только переключать значение**, выставить одновременно 1 на оба входа не получится. Если J и K оба единицы, то значение будет **переключаться ровно 1 раз** при восходящем фронте тактового генератора

Edge detector. Делитель частоты

А как работает детектор переднего фронта?



Делитель частоты



Соединим выходы Set и Reset и подключим их всегда на логическую 1. Теперь при каждом восходящем фронте тактового генератора значение схемы **будет меняться**, а при спаде сигнала значение **меняться не будет**.

Получили **деление частоты** ровно в 2 раза. Эта схема часто используется в устройстве **RCC**

ALU

В разделе C++ мы изучили, как знаковые числа хранятся в памяти ЭВМ. Для этого используется **доп. код**.

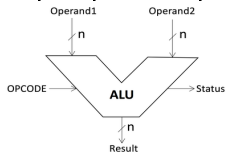
Напомним, что положительные числа хранятся как беззнаковые, отрицательные записываются в память как $\bar{A} + 1$, где дополнение - это инверсия битов.

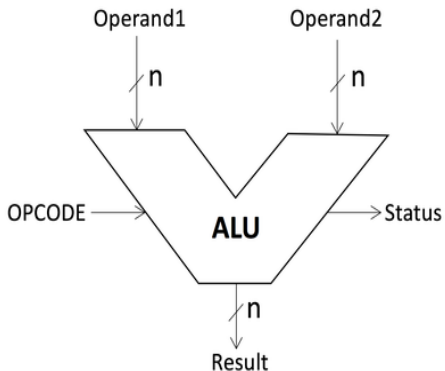
Проверим:

$$(\bar{A} + 1) + A = (\bar{A} + A) + 1 = (2^{16} - 1) + 1 \equiv 0 \pmod{2^{16}}$$

Теперь посмотрим на арифметические операции со стороны хардвара.

ALU - **Arithmetic and Logic Unit** - Арифметико Логическое Устройство. Оно обычно есть в каждом ядре процессора и отвечает за операции над данными.





- ▶ Два **входа** - **левый** и **правый**.
 - На оба входа подаются двухзначные числа - **операнды**.
- ▶ OPCODE - **Operation Code** - Код выбранной операции (И, ИЛИ, +, -, ...).
- ▶ Result - результат операции
- ▶ Status - Системные данные о готовности + **признаки результата**

ALU. Признаки результата NZVC

Специальные биты, значение которых зависит от некоторых событий, которые могут произойти во время вычислений. 1 - Произошло, 0 - нет:

- ▶ N - Negative - результат меньше нуля (угадайте, какой бит нужно проверить, если у STM32 регистры 32-х разрядные).
- ▶ Z - Zero - Результат - 0
 - например при сравнении чисел A и B достаточно подать A - на левый вход, а B на правый, далее вычислить $-B$ в доп коде и сложить $A + (-B)$ - если ALU выставит $Z = 1$ значит A и B равны. (При этом записывать куда-либо результат не нужно, достаточно просто прогнать операцию сложения и посмотреть на бит Z).
 - А как реализовать операции $<, >, <=, >=$?

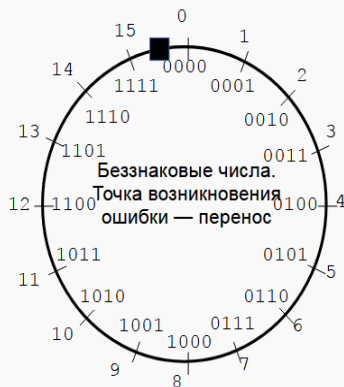
- С - Carry - Перенос. Т.к. разрядная сетка ограничена - если сложить два больших числа результат может не поместиться в 32 бита ячейки памяти. Перенос - точка ошибки только для беззнаковых чисел! Вычисляется, как видно из название, по попытке переноса из старшего разряда в несуществующий, тем самым бит Carry как бы продолжает разрядную сетку регистра.
- Контрпример: Сложите любые $A + (-A) = 0$ - Вы получите 0, но перенос в несуществующий разряд произойдет (через один слайд картинка будет больше)

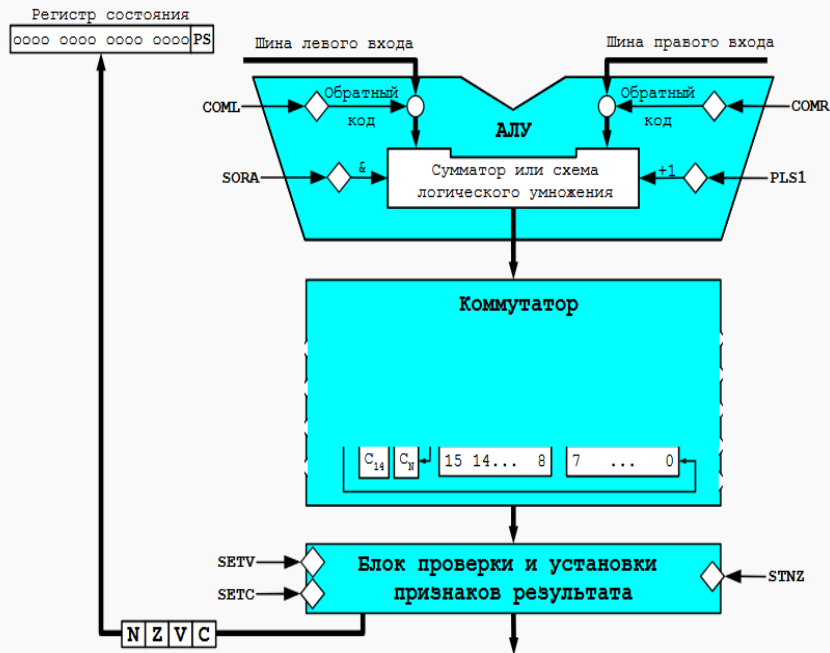


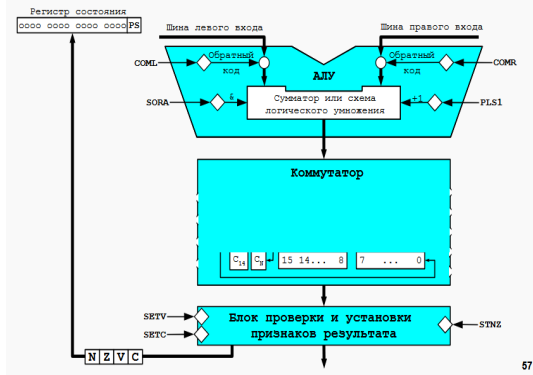
► V - Overflow - Переполнение - ошибка при работе со знаковыми числами. Например если сложить два больших положительных числа, то старший бит c_{31} может стать 1, т.е. число станет отрицательным. Вычисляется через переносы между предпоследним и последним, и последним и carry разрядом $V = C_{31 \rightarrow \text{carry}} \oplus C_{30 \rightarrow 31}$ - подумайте почему именно так.

- Не является ошибкой при работе с беззнаковыми числами.

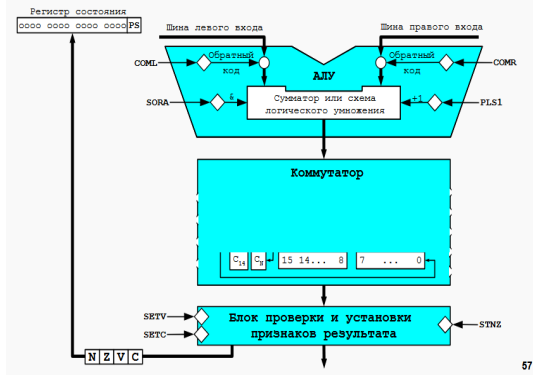






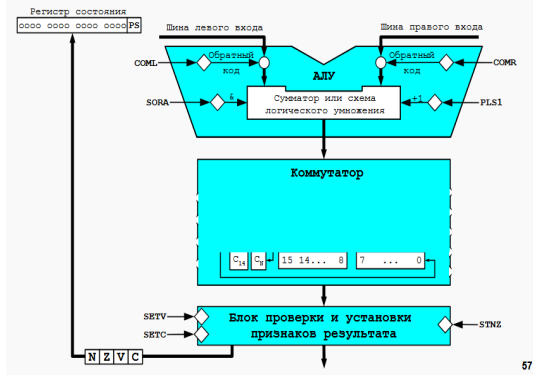


- ▶ Разберем устройство ALU на некотором усредненном примере.
- ▶ Левый и правые входы попадают сначала на генератор обратного (не доп.!!!) кода. Если сигнал **COM(L/R)** 1 - будет инверсия битов, иначе - без изменений.

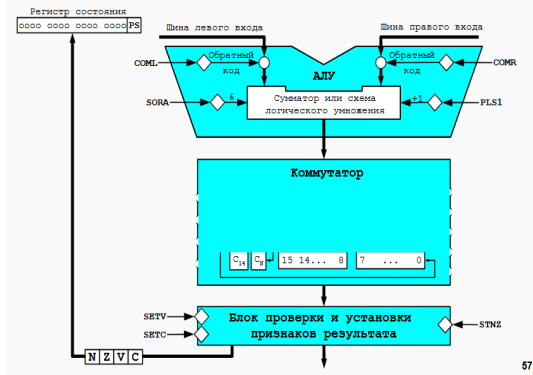


57

- ▶ Далее сигнал приходит на схема сумматора или логического умножения.
 - Если SORA (SUM Or And) 1 - Происходит сумма левого и правого входа, иначе логическое умножение (логическое И)
- ▶ Если PLS = 1 Выполняется операция $R + 1$ - она нужна как раз для вычисления доп. кода числа.



- ▶ Далее вход попадает на Cross Bar Commutator - о его устройстве мы поговорим в главе DMA. Его главная функция - просто направить данные в нужный регистр или устройство.
- ▶ В этом коммутаторе как раз есть 16 битов результата (в STM32 их 32) и бит переноса **V** последний разряд и **ИЗ** последнего разряда (**C₁₄**, **C_{new}** соответственно).



57

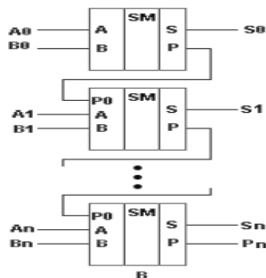
- ▶ Так же в этом коммутаторе формируются признаки результата - биты NZVC. Которые потом идут в обобщенный регистр состояния.
- ▶ Итак, теперь мы знаем как делать +, И, - (1 действие доп. код, 2 - сложение). Логическое действие отдельных схем не действует, достаточно знать закон де Моргана $\neg(\neg A \& \neg B) = A | B$

ALU. Сумматор и логическое умножение

Единственное что мы теперь не знаем - как работает сама схема сумматора и логического умножения.

- ▶ Для этого будем решать задачу поразрядно: Как делать поразрядное логическое И понятно - Соединить одинаковые разряды входных операндов, результат в соответствующий разряд выходного регистра.

- ▶ Для сложения нам понадобится сумматор: Три входа - соответствующие разряды операндов и перенос из предыдущего разряда. Выхода тоже два: значение в разряде результата и перенос в следующий разряд
- ▶ Тогда объединив поразрядные сумматоры - получим сумматор всего числа
- ▶ Чтобы сформировать флаг Carry, откуда надо взять информацию в сумматоре? А между какими сигналами нужно сделать xor ($\oplus$) для V?



Теперь объединим всё в ALU!

Система управления лог. «И»

Схема управления логическим элементом «И» (AND). Входы: 16-битные шины (16). Выходы: 19-битная шина (19). Блок суммирования (Σ 15) и блок AND (& 15) принимают на вход 15-битные шины (15). Блок суммирования (Σ 0) и блок AND (& 0) принимают на вход 0-битные шины (0). Выходы суммирования (Σ 15) и AND (& 15) соединены с 16-битными шинами (16). Выходы суммирования (Σ 0) и AND (& 0) соединены с 0-битными шинами (0). Выходы 16-битных шин (16) соединены с 19-битной шиной (19). Выходы 0-битных шин (0) соединены с 19-битной шиной (19). Выходы 19-битной шины (19) соединены с 19-битной шиной (19).

Легенда:

- PLS1 +1
- SORA &

В этой презентации использованы фрагменты видео с очень интересного канала Core Dumped
<https://youtube.com/@coredumped?si=aH26LLnzdDBBSxPN>

Также использованы слайды курса на кафедре вычислительной техники ИТМО, их можно найти в папке docs в репозитории

Материалы курса можно найти
по ссылке:

<https://github.com/haaroner/ezh239>